

Date

Week - 2

2013

1) Fraud Transaction Detection (Linear Search) :-

\* Problem Statement :- A bank stores transaction IDs in the order they occur, since the transactions are not sorted, the security team needs to check if a suspicious transaction ID exists.

- \* Logic :-
- 1) Read the no. of transaction IDs.
  - 2) Store all transaction IDs in an array.
  - 3) Read the suspicious transaction ID.
  - 4) Traverse the array from start to end.
  - 5) If the target ID matches any element, print "Fraud Transaction Found".
  - 6) Otherwise print "Transaction Not Found".

\* Pseudo Code :-

```

START
Read N
Create array arr of size N
For i = 0 to N-1
    Read arr[i]
Read target
For i = 0 to N-1
    if arr[i] == target
        print "Fraud Transaction Found"
print "Transaction Not Found"

```

\* Java Code :-

```

import java.util.Scanner;

public class Main {
    public static void main (String[] args) {

```

```

Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] arr = new int[n];

```

```

for (int i = 0; i < n; i++) {
    arr[i] = sc.nextInt();
}

```

```

int target = sc.nextInt();
boolean found = false;

```

```

for (int i = 0; i < n; i++) {
    if (arr[i] == target) {
        found = true;
        break;
    }
}

```

```

if (found)
    System.out.println("Found Transaction Found");

```

```

else
    System.out.println("Transaction Not found");
}

```

**Output**

5  
1201  
1305  
1450  
1408  
1502

1450  
Fraud Transaction Found

=== Code Execution Successful ===

Date

2013

2) Server Log Search (Binary Search):-

\* Problem Statement :- A cloud company stores server log IDs in sorted order. Engineers need to quickly verify if a log ID exists.

\* Logic :- 1) Read sorted log IDs.

2) Set low = 0 and high = n-1.

3) Find middle element.

4) If middle equals target → found.

5) If target is smaller → search left half.

6) Else → search right half.

7) Repeat until found or range becomes invalid.

\* Pseudo code :-

Start

Read N

Read sorted array

Read target

low = 0

high = N-1

WHILE low ≤ high

mid = (low + high) / 2

IF arr[mid] == target

Print "Log Found"

ELSE IF target < arr[mid]

high = mid - 1

ELSE

low = mid + 1

Print "Log Not Found"

\* Java Code :-

import java.util.Scanner;

public class Main {

public static void main (String[] args) {

Scanner sc = new Scanner(System.in);

~~int n = sc.nextInt();~~

int[] arr = new int[n];

for (int i = 0; i < n; i++)

arr[i] = sc.nextInt();

int target = sc.nextInt();

int low = 0;

int high = n-1;

boolean found = false;

while (low ≤ high) {

int mid = (low + high) / 2;

if (arr[mid] == target) {

found = true;

break;

} else if (target < arr[mid]) {

high = mid - 1;

} else {

low = mid + 1;

}

} if (found)

System.out.println("Log Found");

else

System.out.println("Log Not Found");

Output

7

1001

1005

1010

1015

1020

1030

1040

1015

Log Found

=== Code Execution Successful ===



Date

2013

### 3) Sort Employee Salaries (Bubble Sort):-

\* Problem Statement :- A company wants to sort employee salaries in ascending order for payroll analysis.

- \* Logic :-
- 1) Compare adjacent elements.
  - 2) Swap if left element is greater than right element.
  - 3) Repeat for all elements.
  - 4) After each pass, the largest element reaches its correct position.
  - 5) Continue until array becomes sorted.

#### \* Pseudocode :-

```

START
Read N
Read array
For i=0 to N-2
    For j=0 to N-2-i
        If arr[j] > arr[j+1]
            Swap arr[j] and arr[j+1]
Print array
    
```

#### \* Java Code :-

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
    
```

```

        int[] arr = new int[n];
        for (int i=0; i<n; i++) {
            arr[i] = sc.nextInt();
        }
        for (int i=0; i<n-1; i++) {
            for (int j=0; j<n-1-i; j++) {
                if (arr[j] > arr[j+1]) {
                    int temp = arr[j];
                    arr[j] = arr[j+1];
                    arr[j+1] = temp;
                }
            }
        }
        for (int num : arr) {
            System.out.print(num + " ");
        }
    
```

### Output

```

5
45000
32000
55000
28000
60000
28000 32000 45000 55000 60000
    
```

=== Code Execution Successful ===

#### 4) Sort Website Response Time (Insertion Sort):-

\* Problem Statement :- A web monitoring system records website response time. New data comes continuously and must be inserted into the correct position.

- \* Logic :-
- 1) Assume first element is already sorted.
  - 2) Pick next element.
  - 3) Compare it with previous elements.
  - 4) Shift larger elements one position right.
  - 5) Insert the current element at correct position.
  - 6) Repeat for all elements.

#### \* Pseudocode :-

```
START
Read N
Read array
For i=1 to N-1
    Key = arr[i]
    j = i-1
    WHILE j >= 0 AND arr[j] > Key
        arr[j+1] = arr[j]
        j--
    arr[j+1] = Key
print array
END
```

#### \* Java Code :-

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
```

```
int[] arr = new int[n];
for(int i=0; i<n; i++) {
    arr[i] = sc.nextInt();
}
for(int i=1; i<n; i++) {
    int key = arr[i];
    int j = i-1;
```

```
while (j >= 0 && arr[j] > key) {
    arr[j+1] = arr[j];
    j--;
}
arr[j+1] = key;
```

```
for(int num : arr) {
    System.out.print(num + " ");
}
```

#### Output

```
5
250
120
320
180
100
100 120 180 250 320
=== Code Execution Successful ===
```

Date \_\_\_\_\_

2013

5) Prioritize Bug Reports (Selection sort):-

\* Problem Statement :- A software company assigns priority scores to bug reports. To resolve bugs efficiently, they want to sort priorities in ascending order.

\* Logic :- 1) Assume first unsorted element is min.  
 2) Find the smallest element in remaining array.  
 3) Swap it with current position.  
 4) Repeat for all positions.  
 5) Array becomes sorted.

\* Pseudocode :-

START

Read N

Read array

For  $i = 0$  to  $N - 2$      $minIndex = i$     For  $j = i + 1$  to  $N - 1$         IF  $arr[j] < arr[minIndex]$              $minIndex = j$     Swap  $arr[i]$  and  $arr[minIndex]$ 

Print array

END

\* Java Code :-

import java.util.Scanner;

public class Main {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

int n = sc.nextInt();

int[] arr = new int[n];

```
for(int i=0; i<n; i++){
    arr[i] = sc.nextInt();
}
```

```
for(int i=0; i<n-1; i++){
    int minIndex = i;
```

```
    for(int j=i+1; j<n; j++){
        if(arr[j] < arr[minIndex]){
            minIndex = j;
        }
    }
```

```
    int temp = arr[i];
    arr[i] = arr[minIndex];
    arr[minIndex] = temp;
```

```
for(int num : arr){
    System.out.print(num + " ");
}
```

}

}

Valvoline

**Output**

5

9

2

8

1

5

1 2 5 8 9

=== Code Execution Successful ===